

Tensorflow II

Digital Image

Fatemeh Dehrouyeh



Contents

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries
 - OpenCV
 - Pillow
 - Matplotlib
 - NumPy
 - scikit-image
 - TensorFlow
 - PyTorch

Current Section

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries

Why Do We Start with Images?

- Humans look at an image and recognize objects such as **cats, cars, people, buildings, and faces**.
- A computer does not see these objects directly.
- Instead, an image is represented as a collection of **numerical pixel values** arranged in a grid.

Key Idea

For a machine learning model, an image is simply a **tensor**: a multidimensional array of numbers.

Image $\longrightarrow$ **Pixels** $\longrightarrow$ **Tensor** $\longrightarrow$ **Neural Network** $\longrightarrow$ **Prediction**

What we see: “*a cat*”

$\neq$

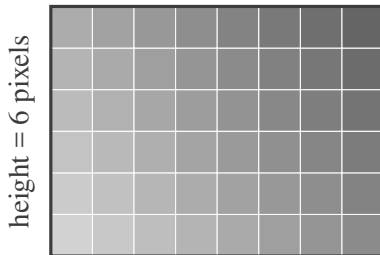
What the computer receives: *numbers*

Current Section

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges**
- 3 Python Image Libraries

A digital image is a grid

- Each small square is a **pixel**.
- Each pixel stores intensity or color values.
- The image has height and width.



height = 6 pixels

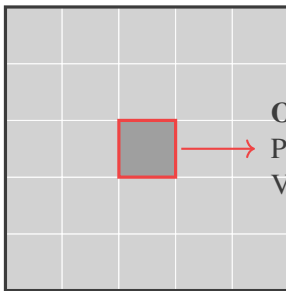
width = 8 pixels

What Is a Pixel?

A **pixel** (picture element) is the smallest addressable location in a digital image.

Each pixel has:

- a position in the image, such as (x, y) ,
- one or more numerical values,
- values that describe intensity or color.



One pixel
Position: (x, y)
Value: 180

For a grayscale image, one pixel may store:

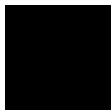
$$I(x, y) = 180$$

This number tells us how bright that pixel is.

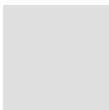
$$\text{Pixel} = \text{Position} + \text{Value}$$

Pixel Values Represent Brightness

For an 8-bit grayscale image, each pixel stores an integer between 0 and 255.



0
Black



64



128
Mid-gray



192



255
White

- Small values correspond to darker pixels.
- Large values correspond to brighter pixels.
- Intermediate values produce different shades of gray.

Key Idea

A grayscale image is simply a grid of these intensity values.

Why Does 8-bit Mean 0–255?

A **bit** can have only two values:

0 or 1

With 8 bits, the number of possible combinations is

$$2^8 = 256$$

Since counting starts from 0, the possible values are

0, 1, 2, ..., 255

Key Idea

An 8-bit grayscale pixel can represent **256 intensity levels**: 0 is black, 255 is white, and the values in between are shades of gray.

Grayscale images

A grayscale image has one intensity value per pixel.

$$\text{shape} = (H, W) \quad \text{or} \quad (H, W, 1)$$

- 0 often means black.
- 255 often means white in 8-bit images.
- Values between 0 and 255 represent shades of gray.

Image resolution tells us how many pixels an image contains, expressed as $\text{Resolution} = \text{Width} \times \text{Height}$, and those pixels are distributed across the image as a rectangular grid.

Grayscale Images: One Number per Pixel



A grayscale image contains only **intensity information**.

A grayscale image is a two-dimensional array:

$$I \in \mathbb{R}^{H \times W}$$

Each pixel contains a single numerical value:

$$I(x, y)$$

For an 8-bit grayscale image:

$$I(x, y) \in \{0, 1, \dots, 255\}$$

- 0 = black
- 255 = white
- values in between = shades of gray

Example

A 512×512 grayscale image contains $512 \times 512 = 262,144$ pixel intensity values.

RGB color images

An RGB image stores three values per pixel:

$$\text{pixel} = [R, G, B]$$



Combined color image

Shape

RGB image: (height, width, 3)

RGB Images: Three Numbers per Pixel



A color image contains multiple **color channels**.

An RGB image is represented as:

$$I \in \mathbb{R}^{H \times W \times 3}$$

Each pixel contains three values:

$$I(x, y) = [R \quad G \quad B]$$

where

- R = red intensity
- G = green intensity
- B = blue intensity

For an 8-bit RGB image:

$$R, G, B \in \{0, \dots, 255\}$$

Example

A 512×512 RGB image has shape $512 \times 512 \times 3$ and contains 786,432 values.

Image dimensions

Most libraries represent an image as:

$$\text{image.shape} = (H, W, C)$$

H height: number of rows

W width: number of columns

C channels: color components per pixel

Example

A color image with shape (224, 224, 3) has 224 rows, 224 columns, and 3 color channels.

RGBA images

Some images have four channels:

$$\text{RGBA} = [R, G, B, A]$$

- The fourth channel is **alpha**.
- Alpha controls transparency.
- Many models expect 3 channels, not 4.

Preprocessing decision

For RGBA images, either remove alpha, convert to RGB, or composite over a background.

Alpha changes how the color is displayed by blending it with the background; the original stored RGB color values do not necessarily change. Note: Compositing an RGBA image over a background means combining the transparent image with a chosen background color before removing the alpha channel. The alpha value controls how much of the foreground pixel and background pixel contribute to the final color: $\text{Final Color} = \alpha(\text{Foreground}) + (1 - \alpha)(\text{Background})$. For example, if $\alpha = 1$, the foreground pixel is fully visible; if $\alpha = 0$, only the background is visible; and if $\alpha = 0.5$, the final pixel is a 50% blend of the foreground and background colors.

Coordinate convention

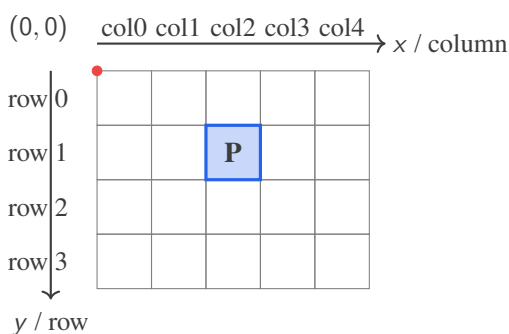
In image processing, coordinates often look different from math plots.

- Image array indexing: `image[row, col]`
- Row corresponds to vertical position: y
- Column corresponds to horizontal position: x
- Top-left corner is usually the origin: $(0, 0)$

Common mistake

Do not confuse `image[x, y]` with `image[y, x]`.

Visualizing Image Coordinates



The highlighted pixel is located at

row = 1, column = 2

Therefore, in an image array:

`image[1, 2]`

Geometrically, the same location is written as:

$(x, y) = (2, 1)$

Remember

Array indexing follows `image[row, col]`, or equivalently `image[y, x]`,
not `image[x, y]`.

Pixel value ranges

Common ranges:

Representation	dtype	Typical range
8-bit image	uint8	0 to 255
Normalized float	float32	0.0 to 1.0
Standardized float	float32	roughly negative to positive

Deep learning habit

Neural networks usually train better with floating-point inputs than raw uint8 values.

Why Normalize Pixel Values?

Images loaded from files often contain values between

0 and 255

Neural networks usually work with floating-point numbers, so we often rescale them:

$$x_{\text{normalized}} = \frac{x}{255}$$

For example, $0 \rightarrow 0.0$ $128 \rightarrow 0.502$ $255 \rightarrow 1.0$

Why do this?

Normalization gives the network inputs on a smaller and more consistent numerical scale, which generally makes optimization and training easier.

Important

Normalization does **not** change what is in the image. It only changes how the pixel values are numerically represented.

Normalization changes the numerical scale of the RGB values while preserving their relative relationship, so the pixel still represents the same color.

Standardizing Pixel Values

Another common preprocessing technique is **standardization**.

Instead of mapping pixel values to a fixed range such as $[0, 1]$, standardization transforms them relative to their mean and standard deviation:

$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

where

- x is the original pixel value,
- μ is the mean pixel value,
- σ is the standard deviation.

After standardization:

mean ≈ 0 and standard deviation ≈ 1

Interpretation

Values below the mean become **negative**,
values near the mean become close to **zero**,
and values above the mean become **positive**.

What Is dtype?

dtype means **data type**. It tells us what kind of numbers are stored in an array or tensor.

- `uint8`: integers from 0 to 255
Common for images loaded from files.
- `float32`: decimal values
Common for normalized images and TensorFlow computations.
- `int32`: integer values
Often used for class labels, indices, or coordinates.

Why does dtype matter?

The data type affects the **allowed values**, **memory usage**, and the mathematical operations we can perform.

Three Questions to Always Ask

What are the image shape, dtype, and value range?

Are Data Type and Value Range Connected?

Yes, but they describe different things.

Data Type (dtype)

The data type determines what kind of numbers can be stored and the **possible numerical range**.

For example:

`uint8` $\Rightarrow$ 0 to 255

Actual Value Range

The actual value range tells us which values are really present in a particular image.

For example, an image with `dtype = uint8` might actually contain only values between 35 and 220.

Important

`dtype` determines the **possible range**, while the value range describes the **actual data**.

Why Do We Care About the Actual Pixel Values?

The dtype tells us what values are **allowed**, but the actual pixel values tell us what information the image is really using.

For example, two images can both have `dtype = uint8`, but their actual ranges may be very different:

Image A: 0 to 255

Image B: 80 to 150

- A narrow range may indicate **low contrast**.
- A very low range may indicate a **dark image**.
- A range such as `[0, 1]` may indicate the image is already **normalized**.
- The range helps us decide what preprocessing is needed.

Key Idea

`dtype` tells us what values are possible; the actual range tells us what values the image is really using.

Current Section

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries**
 - OpenCV
 - Pillow
 - Matplotlib
 - NumPy
 - scikit-image
 - TensorFlow
 - PyTorch

Popular Python Libraries for Working with Images

Python provides several widely used libraries for image processing, visualization, and deep learning.

Library	Main Use	Common Role
OpenCV (cv2)	Image processing	Read, resize, filter, transform
Pillow (PIL)	Image manipulation	Open, save, convert formats
Matplotlib	Visualization	Display images and results
scikit-image	Image processing	Filters, segmentation, examples
NumPy	Numerical arrays	Pixel-level operations
TensorFlow	Deep learning	Tensors, preprocessing, CNNs
PyTorch	Deep learning	Tensors, datasets, vision models

Current Subsection

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries**
 - OpenCV
 - Pillow
 - Matplotlib
 - NumPy
 - scikit-image
 - TensorFlow
 - PyTorch

OpenCV: Image Processing Library

OpenCV stands for **Open Source Computer Vision Library**.

It is one of the most popular Python libraries for working with images and videos.

- Read and save images
- Resize, crop, rotate, and flip images
- Convert between color spaces
- Apply filters and blurring
- Detect edges, contours, and objects
- Process video frames

In Python

OpenCV is usually imported as:

```
import cv2
```

Current Subsection

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries**
 - OpenCV
 - **Pillow**
 - Matplotlib
 - NumPy
 - scikit-image
 - TensorFlow
 - PyTorch

Pillow: Simple Image Manipulation

Pillow is a friendly library for opening, saving, and converting images.

- Open common image formats such as PNG and JPEG
- Save images to disk
- Convert image modes such as RGB, grayscale, and RGBA
- Basic resizing, cropping, and rotation

Import

```
from PIL import Image
```

Current Subsection

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries**
 - OpenCV
 - Pillow
 - **Matplotlib**
 - NumPy
 - scikit-image
 - TensorFlow
 - PyTorch

Matplotlib: Displaying Images

Matplotlib is mainly used for visualization.

- Display images in notebooks and scripts
- Plot histograms of pixel values
- Compare original and processed images
- Visualize training results and model outputs

Import

```
import matplotlib.pyplot as plt
```

Current Subsection

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries**
 - OpenCV
 - Pillow
 - Matplotlib
 - NumPy**
 - scikit-image
 - TensorFlow
 - PyTorch

NumPy: Images as Arrays

NumPy is the foundation for numerical image representation.

- Store images as arrays
- Inspect image shape, dtype, minimum, and maximum values
- Access and modify individual pixels
- Perform mathematical operations on images

Import

```
import numpy as np
```

Current Subsection

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries**
 - OpenCV
 - Pillow
 - Matplotlib
 - NumPy
 - scikit-image**
 - TensorFlow
 - PyTorch

scikit-image: Image Processing Tools

scikit-image provides many image-processing algorithms.

- Built-in example images
- Filtering and denoising
- Segmentation
- Feature extraction
- Measurement and morphology

Import

```
from skimage import data, filters, transform
```

Current Subsection

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries**
 - OpenCV
 - Pillow
 - Matplotlib
 - NumPy
 - scikit-image
 - TensorFlow**
 - PyTorch

TensorFlow: Images for Deep Learning

TensorFlow is used to prepare images for neural networks.

- Load and decode image files
- Convert images to tensors
- Resize and normalize images
- Build image datasets
- Train CNNs and vision models

Import

```
import tensorflow as tf
```

Current Subsection

- 1 What is a digital image?
- 2 Image shape, channels, dtype, and pixel ranges
- 3 Python Image Libraries**
 - OpenCV
 - Pillow
 - Matplotlib
 - NumPy
 - scikit-image
 - TensorFlow
 - **PyTorch**

PyTorch: Another Deep Learning Library

PyTorch is another popular framework for deep learning and computer vision.

- Work with image tensors
- Build datasets and data loaders
- Apply image transformations
- Train CNNs and vision transformers
- Use pretrained vision models

Import

```
import torch
```

Congratulations!

You now know the basics of digital images!

